

HANDS-ON PROJECT • BUILD & LEARN •

Hospital Management System

Database Project

Learn every core database concept by building a real hospital system

Step-by-Step • Full SQL • Sample Data • 20 Practice Exercises

Relationships vs ACID umns Primary Keys Foreign Keys Schema Indexes Constraints

Patients • Doctors • Wards • Appointments • Prescriptions • Lab • Billing • ACID Transactions

Section 1 — Core Concepts

What You Will Build

You will design and build a complete **Hospital Management System** database — a real-world system used to manage patients, doctors, wards, appointments, prescriptions, lab tests, and billing. Every step teaches one or more core database concepts through practical, hands-on SQL.

Phase	Step	What You Build	Concept Learned
1 — Design	Step 1	Identify 11 entities and map relationships	Tables, Schema
2 — Create	Step 2	Create database and parent tables	Columns, Data Types
2 — Create	Step 3	Add Primary Keys to all tables	Primary Key
2 — Create	Step 4	Link tables with Foreign Keys	Foreign Key, Relationships
2 — Create	Step 5	Add constraints for data quality	Constraints (all 6 types)
3 — Populate	Step 6	Insert staff, patients, wards	Rows / INSERT
3 — Populate	Step 7	Insert appointments, prescriptions, labs	Multi-table INSERT
3 — Populate	Step 8	UPDATE and DELETE real scenarios	UPDATE, DELETE
4 — Optimise	Step 9	Create indexes for common lookups	Index, EXPLAIN
5 — Query	Step 10	Write queries across all tables	SELECT, JOINS, Aggregates
6 — Transact	Step 11	Transactions for patient admission & billing	ACID Properties
7 — Practice	Exercises	20 tasks + self-assessment checklist	All Concepts

Schema Design

PHASE 1

Identify real-world hospital entities and plan the database before writing SQL

Concepts covered: Tables • Schema • Relationships (1:1, 1:N, M:N)

Step 1 — Identify Entities (Tables)

A hospital deals with many real-world objects. Each object that needs to be tracked becomes a **table**. Think: "What are the nouns in our system?"

Entity (Table)	What It Stores	Example Data
Departments	Hospital departments	Cardiology, Neurology, Orthopaedics
Doctors	Physicians and specialists	Name, specialisation, dept, salary
Nurses	Nursing staff per ward	Name, shift, ward assigned
Patients	All registered patients	Name, blood group, contact, history
Wards	Hospital wards and rooms	Ward name, type, capacity, floor
Admissions	When a patient was admitted/discharged	Patient, ward, dates, status
Appointments	Patient-doctor consultation slots	Patient, doctor, date, status
Prescriptions	Medicines prescribed per appointment	Medicine, dosage, days, quantity
LabTests	Diagnostic test orders	Test name, patient, ordered by
LabResults	Results against each lab test order	Values, reference range, flag
Billing	Final bill per patient admission	Total charges, paid, due, status

Relationship Map

Table A	Relationship	Table B	Type	FK Implementation
Departments	has many	Doctors	1:N	DeptID FK in Doctors
Departments	has many	Wards	1:N	DeptID FK in Wards
Wards	has many	Nurses	1:N	WardID FK in Nurses
Patients	has many	Appointments	1:N	PatientID FK in Appointments
Doctors	has many	Appointments	1:N	DoctorID FK in Appointments
Appointments	has many	Prescriptions	1:N	AppointmentID FK in Prescriptions
Patients	has many	Admissions	1:N	PatientID FK in Admissions
Wards	hosts many	Admissions	1:N	WardID FK in Admissions
Patients	orders many	LabTests	1:N	PatientID FK in LabTests

Table A	Relationship	Table B	Type	FK Implementation
Doctors	orders many	LabTests	1:N	DoctorID FK in LabTests
LabTests	has one	LabResults	1:1	LabTestID FK UNIQUE in LabResults
Admissions	generates one	Billing	1:1	AdmissionID FK UNIQUE in Billing

Schema Diagram

SCHEMA DIAGRAM

```

[Departments] 1:N [Doctors] 1:N [Appointments] 1:N [Prescriptions]
| |
1:N 1:N
| |
[Wards] 1:N [Nurses] [LabTests] 1:1 [LabResults]
|
1:N (via [Admissions]) < [Patients] 1:N [Appointments]
| |
1:1 (via [Billing]) 1:N > [LabTests]

```


Step 4 — Appointments, Admissions, Prescriptions, Labs, Billing

[illegible]

PHASE 3

Concepts covered: Rows • INSERT INTO • Multi-table Population

```
-- ■■■ Departments ██████████  
INSERT INTO Departments (DeptName, DeptCode, HoD, Floor, Phone) VALUES ('Cardiology',  
'CARD', 'Dr. Ramesh Kumar', 3, '044-2501-1001'), ('Neurology', 'NEUR', 'Dr. Sunita Iyer',  
4, '044-2501-1002'), ('Orthopaedics', 'ORTH', 'Dr. Vijay Menon', 2, '044-2501-1003'),  
(('Gynaecology', 'GYNE', 'Dr. Priya Nair', 1, '044-2501-1004'), ('Paediatrics', 'PAED', 'Dr.  
Anita Sharma', 1, '044-2501-1005'), ('Radiology & Imaging', 'RADI', 'Dr. Kiran Babu', 0,  
'044-2501-1006'), ('Pathology & Lab', 'PATH', 'Dr. Meena Pillai', 0, '044-2501-1007'),  
('Emergency & Trauma', 'EMER', 'Dr. Suresh Rao', 0, '044-2501-1008'); -- ■■ Wards  
██████████████████████████████████████████████████████████████████ INSERT INTO  
Wards (WardName, WardType, Capacity, Floor, DeptID) VALUES ('Cardiac General Ward',  
'General', 30, 3, 1), ('Cardiac ICU', 'ICU', 10, 3, 1), ('Neuro General Ward', 'General',  
25, 4, 2), ('Neuro ICU', 'ICU', 8, 4, 2), ('Ortho Ward', 'Surgical', 20, 2, 3), ('Maternity  
Ward', 'Maternity', 20, 1, 4), ('Paediatric Ward', 'Paediatric', 15, 1, 5), ('Emergency  
Ward', 'General', 20, 0, 8), ('CCU', 'CCU', 6, 3, 1);
```

[illegible]

Step 7 — Appointments, Admissions, Prescriptions, Labs, Billing

Hospital Management System — Hands-On Database Project

```
18000, 8000, 3500, 4800, 700, 0, 'Pending', 'Issued'), (5, 8000, 5000, 1200, 2800, 300,  
17300, 'UPI', 'Paid');
```

Step 8 — UPDATE and DELETE Scenarios

[illegible]

PHASE 4

Concepts covered: Index • EXPLAIN • Query Performance

[illegible]

Index	Column(s)	Query It Serves	Benefit
idx_patient_uhid	UHID (UNIQUE)	Reception finds patient on every visit	Instant lookup, O(1)
idx_patient_phone	Phone	Emergency staff searches by phone	Fast equality search
idx_app_doc_date	DoctorID + AppDate	Doctor's daily appointment list	Avoids full Appointments scan
idx_admission_ward	WardID + Status	How many beds free in a ward?	Direct range scan per ward
idx_bill_status	BillStatus	Finance: all unpaid bills today	Filters millions of bills fast
idx_lab_patient_status	PatientID + Status	All pending labs for one patient	Composite index serves both
idx_doc_spec FULLTEXT	Specialisation	Search 'cardiac' → list cardiologists	Word-level index, no LIKE scan

Query the Database

PHASE 5

SELECT queries — simple, JOIN, aggregate, subquery, window functions

Concepts covered: SELECT • JOIN • GROUP BY • HAVING • Subqueries

Step 10 — Essential Hospital Queries

```
-- Q1: All currently admitted patients with ward and doctor details SELECT p.UHID,
p.FullName, w.WardName, w.WardType, a.BedNo, d.FullName AS AdmittingDoctor, a.AdmittedOn,
a.Diagnosis FROM Admissions a JOIN Patients p ON p.PatientID = a.PatientID JOIN Wards w ON
w.WardID = a.WardID LEFT JOIN Doctors d ON d.DoctorID = a.AdmittingDocID WHERE a.Status =
'Admitted' ORDER BY a.AdmittedOn; -- Q2: Doctor's full schedule for a specific date SELECT
d.FullName, d.Specialisation, a.AppTime, a.Type, p.FullName AS Patient, a.Status, a.Notes
FROM Appointments a JOIN Doctors d ON d.DoctorID = a.DoctorID JOIN Patients p ON p.PatientID
= a.PatientID WHERE a.DoctorID = 1 AND a.AppDate = '2024-06-10' ORDER BY a.AppTime; -- Q3:
Complete patient history – appointments + admissions + labs SELECT p.UHID, p.FullName,
p.BloodGroup, COUNT(DISTINCT a.AppointmentID) AS TotalAppointments, COUNT(DISTINCT
adm.AdmissionID) AS TotalAdmissions, COUNT(DISTINCT lt.TestID) AS LabTestsOrdered,
MAX(a.AppDate) AS LastVisit FROM Patients p LEFT JOIN Appointments a ON a.PatientID =
p.PatientID LEFT JOIN Admissions adm ON adm.PatientID = p.PatientID LEFT JOIN LabTests lt ON
lt.PatientID = p.PatientID GROUP BY p.PatientID, p.UHID, p.FullName, p.BloodGroup ORDER BY
TotalAdmissions DESC;
```

Aggregate and Finance Queries

```
-- Q4: Ward occupancy – beds used vs total capacity SELECT w.WardName, w.WardType,
w.Capacity, COUNT(a.AdmissionID) AS BedsOccupied, w.Capacity - COUNT(a.AdmissionID) AS
BedsAvailable FROM Wards w LEFT JOIN Admissions a ON a.WardID = w.WardID AND a.Status =
'Admitted' WHERE w.IsActive = TRUE GROUP BY w.WardID, w.WardName, w.WardType, w.Capacity
ORDER BY BedsAvailable ASC; -- Q5: Department revenue summary (billing charges) SELECT
dep.DeptName, COUNT(DISTINCT b.BillID) AS TotalBills, SUM(b.TotalAmount) AS TotalCharged,
SUM(b.PaidAmount) AS TotalCollected, SUM(b.TotalAmount - b.PaidAmount) AS Outstanding FROM
Departments dep JOIN Doctors doc ON doc.DeptID = dep.DeptID JOIN Admissions adm ON
adm.AdmittingDocID = doc.DoctorID JOIN Billing b ON b.AdmissionID = adm.AdmissionID GROUP
BY dep.DeptID, dep.DeptName ORDER BY TotalCharged DESC; -- Q6: Patients with abnormal lab
results (need follow-up) SELECT p.UHID, p.FullName, p.Phone, lt.TestName, lr.ResultValue,
lr.ReferenceRange, lr.ReportedOn FROM LabResults lr JOIN LabTests lt ON lt.TestID =
lr.TestID JOIN Patients p ON p.PatientID = lt.PatientID WHERE lr.IsAbnormal = TRUE ORDER BY
lr.ReportedOn DESC; -- Q7: Doctors ranked by number of patients treated SELECT d.FullName,
d.Specialisation, dep.DeptName, COUNT(DISTINCT a.PatientID) AS UniquePatientsOPD,
COUNT(a.AppointmentID) AS TotalAppointments FROM Doctors d LEFT JOIN Appointments a ON
a.DoctorID = d.DoctorID AND a.Status = 'Completed' LEFT JOIN Departments dep ON dep.DeptID =
d.DeptID GROUP BY d.DoctorID, d.FullName, d.Specialisation, dep.DeptName ORDER BY
TotalAppointments DESC;
```

Transactions & ACID

PHASE 6

Critical hospital operations must be atomic, consistent, isolated, and durable

Concepts covered: ACID • BEGIN • COMMIT • ROLLBACK • SAVEPOINT

Step 11 — Transaction 1: Patient Admission (Atomicity)

Admitting a patient involves multiple tables. If any step fails, **all changes must be undone** — a half-admitted patient is dangerous.

```
START TRANSACTION; -- Step A: Register new patient INSERT INTO Patients (UHID, FullName,
DateOfBirth, Gender, BloodGroup, Phone, EmergencyContact) VALUES ('HMS20240009','Rajesh
Pillai','1972-08-15','M','AB-','9800001109','9800001209'); SET @new_patient =
LAST_INSERT_ID(); -- Step B: Create emergency appointment INSERT INTO Appointments
(PatientID, DoctorID, AppDate, AppTime, Type, Status, Notes) VALUES (@new_patient, 8,
CURRENT_DATE, CURRENT_TIME, 'Emergency', 'Completed', 'Road accident – chest trauma'); SET
@new_app = LAST_INSERT_ID(); -- Step C: Admit to Emergency Ward (WardID=8) INSERT INTO
Admissions (PatientID, WardID, AdmittingDocID, BedNo, Diagnosis, Status) VALUES
(@new_patient, 8, 8, 'EM-06', 'Chest trauma – rib fractures', 'Admitted'); SET @new_adm =
LAST_INSERT_ID(); -- Step D: Create draft bill immediately INSERT INTO Billing
(AdmissionID, BillStatus) VALUES (@new_adm, 'Draft'); COMMIT; -- All 4 steps saved together
-- If bed EM-06 is already occupied (UNIQUE constraint on BedNo+WardID), -- Step C fails →
ROLLBACK → patient NOT registered, app NOT created -- The system stays consistent
```

Transaction 2: Bill Payment with Concurrent Safety (Isolation)

```
START TRANSACTION; -- Lock the bill row to prevent concurrent double-payment SELECT
BillStatus, TotalAmount, PaidAmount FROM Billing WHERE BillID = 2 FOR UPDATE; -- row-level
lock: other sessions must wait -- Verify bill is not already fully paid -- (Application
checks: if BillStatus = 'Paid' → ROLLBACK) -- Process payment of Rs. 26,000 UPDATE Billing
SET PaidAmount = PaidAmount + 26000, PaymentMode = 'Insurance', BillStatus = CASE WHEN
PaidAmount + 26000 >= TotalAmount THEN 'Paid' ELSE 'Partially Paid' END WHERE BillID = 2; --
Generate payment receipt log INSERT INTO PaymentReceipts (BillID, AmountPaid, PaymentMode,
TxnRef, ReceivedOn) VALUES (2, 26000, 'Insurance', 'INS2024061400123', NOW()); COMMIT; --
Durable – payment recorded even if server crashes now
```

Transaction 3: Medication Dispensing with SAVEPOINT (Consistency)

```
START TRANSACTION; -- Dispense medicine 1: Aspirin INSERT INTO MedicineDispensed
(AdmissionID, MedicineName, Qty, DispensedOn) VALUES (2, 'Aspirin 75mg', 1, NOW()); UPDATE
MedicineInventory SET StockQty = StockQty - 1 WHERE MedicineName = 'Aspirin 75mg'; SAVEPOINT
aspirin_done; -- Dispense medicine 2: Clopidogrel INSERT INTO MedicineDispensed
(AdmissionID, MedicineName, Qty, DispensedOn) VALUES (2, 'Clopidogrel 75mg', 1, NOW());
UPDATE MedicineInventory SET StockQty = StockQty - 1 WHERE MedicineName = 'Clopidogrel
75mg'; SAVEPOINT clopi_done; -- Dispense medicine 3: Heparin infusion UPDATE
MedicineInventory SET StockQty = StockQty - 2 WHERE MedicineName = 'Heparin 5000 IU'; --
Suppose CHECK triggers: StockQty cannot go below 0 → ERROR ROLLBACK TO SAVEPOINT
clopi_done; -- undo only Heparin step -- Aspirin and Clopidogrel kept; Heparin inventory not
touched -- Alert pharmacy to restock Heparin; use alternate anticoagulant INSERT INTO
MedicineDispensed (AdmissionID, MedicineName, Qty, DispensedOn) VALUES (2, 'Enoxaparin
```

```
40mg', 1, NOW()); COMMIT;
```

Practice Exercises

PHASE 7

20 tasks covering every concept — try them on your HMS database!

Concepts covered: All Concepts • Self-Assessment

Table & Schema Tasks

1 Add new tables to the HMS

- Create a 'Staff' table for non-medical staff (admin, housekeeping) with a WardID FK
- Create an 'OperationTheatre' table and link it to Admissions (1:1 optional)
- Create an 'Ambulance' table and a 'AmbulanceCalls' table (patient, doctor, pickup)

2 Schema modifications

- Add a 'InsuranceProvider' column to Patients (VARCHAR, nullable)
- Add a 'RoomType' ENUM('General','Semi-Private','Private') to Wards with DEFAULT
- Modify Billing to add a DiscountAmount column with CHECK(DiscountAmount >= 0)

Row Manipulation Tasks

3 INSERT tasks

- Add 2 new patients — one for Cardiology, one for Paediatrics
- Schedule 3 appointments for Dr. Sunita Iyer on 2024-06-15 at different times
- Add lab test orders for the new patients and then insert their results

4 UPDATE tasks

- Discharge patient AdmissionID=3 — set DischargedOn to today, update Status
- Promote Dr. Lakshmi Devi to 'Associate Professor' and give a 15% salary raise
- Mark all 'Ordered' lab tests older than 24 hours as 'Processing'

5 DELETE tasks

- Cancel AppointmentID=7 (no-show) — hard delete. Does CASCADE remove prescriptions?
- A nurse resigned — soft delete vs hard delete: which is better here and why?
- Remove all Draft bills for patients who never got admitted (cleanup task)

Constraint Tasks

6 Test and add constraints

- Try inserting a patient with BloodGroup='XY' — what error occurs?
- Try setting Billing.PaidAmount = -500 — which constraint blocks this?
- Try inserting a LabResult with a TestID that doesn't exist in LabTests
- Add a UNIQUE constraint: a doctor can't have two appointments at same date+time

Index Tasks

7 Index optimisation

- Run EXPLAIN on: `SELECT * FROM Prescriptions WHERE AppointmentID = 1`
- No index exists on AppointmentID in Prescriptions — create one and re-check EXPLAIN
- Should you index the Gender column in Patients? Why or why not?

Query Tasks

8 Write these queries

- List all prescriptions for a specific patient across all their appointments
- Find doctors with no appointments in the last 30 days (use NOT EXISTS)
- Show monthly admission count for the current year with total revenue per month
- Find the ward with the highest current occupancy percentage

9 Advanced queries

- Find patients who visited more than one department in the same month
- List the top 3 most prescribed medicines across all appointments
- Show each patient's total amount billed vs total amount paid (outstanding balance)

Transaction Tasks

10 ACID scenarios

- Write a transaction to transfer a patient from Emergency Ward to Cardiac ICU
- Simulate double-payment on same bill — show how FOR UPDATE prevents it
- Write a SAVEPOINT transaction for ordering 4 lab tests — rollback if 3rd test fails
- Discharge a patient: update Admissions, finalize Billing, all in one transaction

Self-Assessment Checklist

PRACTICE CHECKLIST

- ☐ Create the full HMS schema (11 tables) with correct FK order
- ☐ Define columns with all 6 constraint types on at least one table
- ☐ Create 1:N relationship (Departments → Doctors) with CASCADE rules
- ☐ Create 1:1 relationship (LabTests → LabResults) using UNIQUE FK
- ☐ Create M:N-style link (Patients → Doctors via Appointments)
- ☐ Insert at least 5 patients, 5 doctors, 5 appointments, 5 admissions
- ☐ Write an UPDATE affecting multiple rows (e.g. discharge all ICU patients of a date)
- ☐ Write a DELETE with a JOIN condition (remove old cancelled appointments)
- ☐ Use EXPLAIN to confirm an index is being used on a SELECT query
- ☐ Write a 4-table JOIN query (Patient + Admission + Ward + Doctor)
- ☐ Write a GROUP BY with HAVING (wards with occupancy > 50%)
- ☐ Use a correlated subquery (patients whose bill total > average)
- ☐ Wrap patient admission into a BEGIN/COMMIT transaction (4 INSERT steps)
- ☐ Trigger a ROLLBACK and verify all 4 steps were undone with SELECT
- ☐ Use SAVEPOINT for medicine dispensing — partial rollback on stock failure

Quick Reference — All 11 Tables at a Glance

Table	Primary Key	Foreign Keys	Key Constraints	Relationship
Departments	DeptID AUTO	—	DeptName UNIQUE, DeptCode UNIQUE	1:1 Depts
Wards	WardID AUTO	DeptID → Departments	WardName UNIQUE, Capacity>0	N:1 Departments
Doctors	DoctorID AUTO	DeptID → Departments	Email UNIQUE, Phone UNIQUE, Salary>0	1:1 Depts
Nurses	NurseID AUTO	WardID → Wards	Email UNIQUE, Salary≥20000	N:1 Wards
Patients	PatientID AUTO	—	UHID UNIQUE, Gender CHECK, BloodGroup CHECK	1:1 Patients
Appointments	AppointmentID AUTO	PatientID → Patients DoctorID → Doctors	Status ENUM	M:N Patients-Doctors
Prescriptions	PrescID AUTO	AppointmentID → Appointments	DurationDays>0, Qty>0	N:1 Appointments
Admissions	AdmissionID AUTO	PatientID, WardID, DoctorID	DischargedOn>AdmittedOn CHECK	1:1 Each
LabTests	TestID AUTO	PatientID, DoctorID → Doctors	Priority ENUM, Status ENUM	N:1 Patients
LabResults	ResultID AUTO	TestID → LabTests (UNIQUE)	UNIQUE TestID → 1:1	1:1 LabTests
Billing	BillID AUTO	AdmissionID → Admissions (UNIQUE)	UniqId>0, GENERATED TotalAdmissions	1:1 Admissions

Concept Map — What Each Phase Taught You

Concept	Phase/Step	Specific Thing You Did
Table	Phase 1	Identified 11 hospital entities; each became a table
Schema	Phase 1+2	Drew relationship map; wrote full CREATE DATABASE script
Column	Phase 2	Defined columns with types for every table (ENUM, DECIMAL, TEXT...)
Primary Key	Phase 2	AUTO_INCREMENT PK on all tables; UHID natural key for Patients
Foreign Key	Phase 2	12 FK relationships; all cascade rules defined and tested
Constraints	Phase 2+3	All 6 types: PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT
Rows	Phase 3	Inserted 8 patients, 8 doctors, 9 wards, 5 admissions, bills, labs
Index	Phase 4	8 indexes including composite and FULLTEXT; tested with EXPLAIN
Relationships	Phase 2	1:N (Depts→Doctors), 1:1 (Tests→Results), M:N (Patients↔Doctors)
ACID	Phase 6	3 transactions: admission (Atomicity), payment (Isolation), meds (SAVEPOINT)

Extend the Project

Extension	New Tables	Concepts Practised
Blood Bank	BloodBankInventory, BloodRequest	CHECK on blood group compatibility; triggers
Pharmacy Module	MedicineInventory, Dispensing, Reorder	CHECK (StockQty>=0); automated reorder trigger

Extension	New Tables	Concepts Practised
Staff Roster	Shifts, AttendanceLog, Payroll	Composite PK; scheduled jobs
Telemedicine	VideoConsult, ChatLog, ePrescription	LOB for chat; JSON for structured data
Insurance Claims	InsurancePolicies, Claims, ClaimItems	Many FK; status workflow constraints
Reporting Dashboard	(Views + CTEs + Aggregates)	CREATE VIEW; WITH clauses; indexed views